

CT1 with GNU assembly syntax

STUDENTS: To be programmed/END: To be programmed in source code files.

To create a **release** for the students repository, use the prepared bash script. It will clone the students repository and copy the required, adjusted resources into said repository. This ensures, that no (properly tagged) solutions are pushed, and improves release consistency. Note that committing and tagging has to be done manually.

Table of contents

- [CT1 with GNU assembly syntax](#)
- [Table of contents](#)
- [Setting up local repository](#)
- [Installing the required tools](#)
 - [Linux](#)
 - [Windows](#)
 - [Installing MSYS2](#)
 - [Installing Visual Studio Code](#)
 - [Installing tools via MSYS2](#)
 - [Git for Windows](#)
- [Testing setup](#)
- [Workflow with Makefile](#)
 - [Debugging with GDB](#)
- [Workflow with Visual Studio Code](#)
 - [Debugging with Visual Studio Code](#)

Setting up local repository

First clone this repository **locally**. Make sure this directory is **not** on [OneDrive](#) or any other network file system. Keil will not work otherwise and even if you do not using Keil, it is always a potential source of problems.

It is a good idea to make a fork of this repository. If you do so, make sure to keep the repository private, so other students can find the repository using the search function.

Note: You can register your public ssh key with your github profile for easier and more secure access (pull, push, etc.) to repositories.

Then get the `libctboard`, which contains the HAL (hardware abstraction layer) and startup code. One can just download the latest [release](#). It is available as both tarball (for unix-like systems) and zip file (for windows systems). Alternatively one can clone repository and build the library locally.

Place the archive (`.tar.xz` or `.zip`) in the directory `./lib` and extract it there. The `./lib` directory should now contain directories containing the header files, the startup routine, the linker script and the archive with the precompiled object files (`libctboard.a`).

Installing the required tools

The CT1 laboratories can be done on the operating system of choice, as long as the following tools are available:

- [Git](#)
- [Bash](#)
- [GNU Make](#)
- [openocd](#)
- [arm-none-eabi toolchain](#)

One can choose to use the Makefile for flashing and debugging, or an integrated workflow with an IDE. See section [Setup Visual Studio Code](#) for a guide on how to setup visual studio code as IDE for the labs.

Authors choice is Makefiles and GDB, as IDEs are big in scale and therefore harder to debug, automate and are prone to vulnerabilities.

Linux

`git`, `bash` and `make` (GNU Make) are packaged and/or shipped with the base system for all common purpose distributions, so one can just use the systems package manager to install `git`, `bash` and `make`. If one uses a distribution, that does not come with these tools preinstalled and does not provide packages, the path of pain was chosen deliberately..

`openocd` is relatively popular and might be packaged for your system, if not you follow the instructions on their [website](#) or if the website is not reachable try the [official repository](#). When using an exotic Linux distribution, one might have to compile from sources, again everyone chooses their one battles.

Note: If installing from sources, one might need to manually install udev rules.

The `arm-none-eabi toolchain` is available as one or multiple packages on many popular distributions like Arch, Debian and Fedora. If the toolchain is not packaged, one can download the binaries from [ARMs download page](#), ensure to download the one with `arm-none-eabi` in the name. The path to the downloaded binaries needs to be added to the `PATH` variable, in order for `bash` to find the executables. This can be achieved with the command `export PATH=$PATH:<path/to/toolchain/bin>`. Add this command to `~/.bashrc` to make it permanent.

Important: The `arm-none-eabi toolchain` comprises `gcc`, `gdb`, `binutils` and `newlib`, depending on the distribution, these might be separate packages and you need all of them.

Windows

Installing MSYS2

MSYS2 (Minimal System 2) is a software distribution and a development platform for Microsoft Windows, based on Mingw-w64 and Cygwin, that helps to deploy code from the Unix world on Windows.

Download the installer from the [MSYS2 website](#).

During installation, use the default installation directory `C:\msys64`. Using a different installation directory may cause path-related issues later.

Installation path of MSYS2

After the installation has completed, the checkbox to start MSYS2 can be unchecked.

Installation finished of MSYS2

Installing Visual Studio Code

Installing VSCode is optional, you can also work in the terminal. However it does have a decent debugger integration, so it might be worth if you are not familiar with GDB and do not want to put in the effort to learn it. If you choose to use VSCode, be careful with extensions. There have been more attacks through VSCode extensions than I want to count just this year (2026-05-27).

Install [Visual Studio Code](#) if it is not already installed.

After installing Visual Studio Code setup the provided profile `common/ct.code-profile`. Start Visual Studio Code, click the settings button in the bottom-left corner, and select *Profiles*.

VSCode profile settings

After the Profiles window has opened, click the arrow next to the *New Profile* button and select *Import Profile...*

VSCode profile import

Navigate to the local repository and select `laboratories/common/ct.code-profile`.

VSCode profile path

Before creating the profile, make sure that the settings and extensions from the CT profile are selected for import. The other options can be chosen as needed.

VSCode profile create

After clicking *Create*, Visual Studio Code imports the selected settings, such as the default terminal configuration, and installs the required extensions.

Now activate the newly created profile.

Installing tools via MSYS2

After Visual Studio Code has been configured, open the integrated terminal with `Ctrl+Shift+!`. Visual Studio Code should automatically open the MSYS2 UCRT64 terminal. Alternatively you can always use `Ctrl+Shift+p` to open the fuzzy finder and search for terminal.

Note: If a folder is opened in Visual Studio Code, the integrated terminal opens in that folder by default. For the MSYS2 setup, open the `laboratories` folder in Visual Studio Code.

If MSYS2 has been newly installed, a system upgrade needs to be performed first. Run the command below:

```
1 pacman -Syu
```

MSYS2 system upgrade

If asked to proceed with the installation, accept. When no input is given (just hitting enter) the capital option is used.

During the upgrade, `pacman` may ask to close the terminal to finish the process. Accept again.

MSYS2 terminal kill

After the terminal has closed, reopen it. To install the required tools, a bash script is provided in `./common`.

Run the command below from the `laboratories` directory:

```
1 ./common/msys_init.sh
```

MSYS2 bash script

After the installation has completed, compiling, flashing, and debugging are ready to use.

Git for Windows

The setup script for MSYS2 will install Git for the MSYS bash shell. If you want fancy Git integration in VSCode you either need to figure out the path setup, or install Git a second time via [Git for Windows](#), below is a guide for the latter.

The picture below shows the component selection with the required options. Additional components can be selected if needed for other purposes.

components to install

An editor has to be selected during installation. Since Visual Studio Code is already installed, it can be selected as the editor. Alternatively, editors such as Vim can be used, the default is vi, which might not even be installed.

git editor selection

If needed, Git Credential Manager can be enabled. When enabled, Git can store login credentials, so they do not have to be entered every time a push or pull requires authentication.

A common alternative is to use an SSH key with no password. This is not recommended, but many developers do it anyways.

git credentials option

Testing setup

At this point the local repository should be setup, so `git` is working.

Since you probably used `git` in `bash`, it is probably installed correctly too.

If `./lib/` was populated, one can open laboratory `01-target-system` and try building it. If it builds without errors, `make` and the compiler work as intended.

To test `openocd`, connect a CT board and try flashing lab 1.

Workflow with Makefile

When working with the Makefile, one can edit the sources with the editor of choice and compile, flash and debug with the make targets. The following targets are defined:

target	description	command
default, all	incremental build	<code>make, make all</code>
clean	delete all build artifacts	<code>make clean</code>
rebuild	clean build	<code>make rebuild</code>
flash	flash binary to the board	<code>make flash</code>
dbgsrv	start debug server	<code>make dbgsrv</code>
launchgdb	start debugger	<code>make launchgdb</code>

Note: In order to use the makefile one needs to navigate to the makefiles directory.

Debugging with GDB

To debug with GDB you first need to start the debug server. We use `stutil` to host a local debug server for a remote target. It can be started with the Make target `dbgsrv`. Next launch the client in a separate shell instance with the Make target `launchgdb`, this will open GDB as interactive command line interface.

freshly opened GDB

The CLI is used to interact with GDB, with the `layout` command you can enable for example the `src` window, which displays the source code. When enabling a **layout** usually the focus is on one of the new windows, which may lead to unexpected behaviour of keys like the arrows. To change **focus** to another window, use the `focus [asm|cmd|regs|src]` command.

Window	Description
<code>asm</code>	shows the disassembly of the program
<code>cmd</code>	interactive window used to control GDB, but also print output of commands like <code>info</code> , <code>print</code> , <code>x</code>
<code>regs</code>	shows the contents of the processors register
<code>src</code>	displays source code, by default at the point of execution

NOTE: For the labs the custom layout `ines` is provided. It is defined in `./common/.gdbinit`.

GDB layout ines

NOTE: GDB supports completions with `tab`.

You can exit GDB using the command `quit`.

To set **breakpoints** use `breakpoint <position>` or just `b <position>`. See the table on the position argument below for details on how to specify the position for a breakpoint. Use `info breakpoints` to list all breakpoints. Delete breakpoints with `delete <breakpoint-number>`.

To **continue** normal execution after a reset, or breakpoint use `continue` (or just `c`). `step`, `next` and `finish` are the equivalents to **step** in/over/out.

- *Step in* executes just one instruction (or line in case of a higher level language), it enters functions if encountered.
- *Step over* executes the next instruction, does not enter functions.
- *Step out* executes instructions until the current function returns.

Use `x` (examine) to **view memory** contents.

```
1 # synopsis
2 x[/format] <address>
3
4 # example without format
5 x 0x20000000
6
7 # example with format
8 # prints 4 bytes starting at 0x2000_0000 in hexadecimal format
9 x/4xb 0x20000000
10
11 # format:
12 # x/nfs
13 #
14 # n = number of elements to print
15 #
16 # f = format to print:
17 #     o = octal
18 #     x = hexadecimal
19 #     d = decimal
20 #     u = unsigned decimal
21 #     t = binary
22 #     f = float
23 #     a = address
24 #     i = instruction
25 #     c = char
26 #     s = string
27 #     z = hex, zero padded on the left
```

```

28 #
29 # s = size of one element:
30 #     b = byte (8 bits)
31 #     h = halfword (16 bits)
32 #     w = word (32 bits)
33 #     g = giant (64 bits)

```

position	description
<symbol-name>	name of a symbol, e.g. function name
<line>	line number (in current file)
<file>:<line>	line number in the specified file

Workflow with Visual Studio Code

When working with Visual Studio Code, make sure to add only the current lab directory to the workspace, not the directory containing all labs. Otherwise, Visual Studio Code might not detect the Makefile correctly.

To compile or flash the program, use Visual Studio Code's task system. Tasks can be launched by pressing **Ctrl+Shift+B** and selecting the desired task.

Note: When multiple labs are open in the workspace, make sure to run the task for the lab currently being worked on.

Note: There is a Vim extension for Visual Studio Code, which can make editing more comfortable for Vim users.

Debugging with Visual Studio Code

To start the debugger, open the debugging view by clicking the debug icon or by pressing **Ctrl+Shift+D**. Select the desired target next to the green play button, then press the play button or **F5**.

Debug Window

Note: Registers and variables are only updated when the program is paused.